BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Explicit inputs, constraints and a testable outcome.

Q2. A representation keeping relevant detail and omitting irrelevant detail.

Q3. Named subroutine performing a task.

Q4. D, B, A. C is popped and D is pushed above B.

You Do pages · Independent questions

Q1. Validate lateDays→valid/invalid [2]; CalculateFine→amount [2]; DisplayResult→message [2].

Q2. if(daysLate<=0)return0 [2], daysLate*0.20m [2], Math.Min(2m,...) [2].

Q3. Separately testable components [2], parallel development/reuse [2], integration/interface mismatches or coordination overhead [2].

Exit Ticket · Model answer

False. Interfaces and integration are essential.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

Do Now — Recall previous learning | I Do — Read new knowledge | We Do — Practise with guidance | You Do — Work independently | Review & Improve — Check understanding

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 066 · Lesson 066 · Problem Decomposition: Build a Useful Solution

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What makes a problem suitable for computation?

Q2. What is an abstraction?

Q3. What is a procedure?

Q4. Stack initially empty: Push A, B, C; Pop; Push D. Give the stack top-to-bottom.

Do Now — Recall previous learning | I Do — Read new knowledge | We Do — Practise with guidance | You Do — Work independently | Review & Improve — Check understanding

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

decomposition — Split task into smaller problems.

top-down design — Start broad, refine into subtasks.

interface — Inputs/outputs between components.

modularity — Small cohesive program components.

New learning

Define and predict — Decomposition divides a larger problem into manageable sub-problems whose inputs and outputs are clear.

Explain — A top-down design begins with a main task, then refines it into independently testable functions or procedures.

Evaluate — Interfaces between subproblems matter: validation should happen before data are stored or a fee is charged.

Worked example

Decompose a cinema price calculator for child/adult tickets.

1. Follow the reasoning — State the interface for each subtask; every step has a specific responsibility.

We Do · Practise with guidance

Decompose a sports-club booking app into four named operations.

```
ValidateMember → CheckPlaces → ReservePlace → SendConfirmation.
```

Follow the reasoning

Reservation must occur before confirming success; check for failures.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Identify three independent subtasks and their outputs for a library fine system. [6]

Question 2

Write C# Fine(daysLate) for 20p/day capped at £2, zero for on-time returns. [6]

Question 3

Explain two benefits and one limitation of decomposition when a team builds a school booking app. [6]

Exit Ticket · Answer before checking

Splitting work means the components never need to communicate.